should be careful about how you implement this operation).

#6 If there is an outgoing edge in `node` for the character `c`, we can keep recursively traversing the tree.

#7 Recursively call delete on `tail`, and save the result.

#8 If the key was removed, and the next node in the path now can be pruned…

#9 …first, remove the edge for character `c`…

#10 Then, if this node is either a key node or an inner node (i.e. if it does have at least another child left)…

#11 …it means that this node can't be pruned anymore, so we update the flag, before returning.

#12 If execution gets here, it means we haven't found the key in the trie, and so it can't be deleted.

This version of removal obviously needs a different implementation of the trie method than the naïve one shown in listing 6.7; in this case, though, the API method is even simpler than the shown in listing 6.7, basically it just becomes a wrapper.

Performance-wise, the same considerations made for `search` and `insert` apply for `remove`; if we implement the method with pruning, the number of operations on the tree is at worst `2*m`, twice as much as for the naïve version without pruning, for which at most `m` edges are traversed; execution time is also probably going to be more than 2-fold, because, as you can see from code, the delta in code complexity is relevant.

The tradeoff for faster running time, however, is that the tree can grow significantly if we don't prune dangling branches;